

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	ATHIN SIVA.S	Reg. No.:	192524237
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big- Θ , Big- Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					



SIMATS Online Programming Lab
DAA PROGRAMMING


Athin Siva S
192524237RC

My Assessments
Course: CSA0612RC

COS AT2-Pseudocode Writing Task (Algorithm Design)
OPEN
✓ Completed

1/1 answered

Review →

View Rubric

Criteria	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Problem Understanding 20%	Identifies all inputs, outputs, and constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design 30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures 20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure 10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness 20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Problem Statement

Sudoku is a constraint-satisfaction problem on a 9×9 grid divided into nine 3×3 subgrids. Some cells contain digits 1 to 9 and the remaining cells are empty. The objective is to fill every empty cell so that each digit appears exactly once in every row, column, and 3×3 subgrid.

Backtracking solves the puzzle by filling one empty cell at a time. Before placing a digit, the algorithm validates all Sudoku constraints. If a partial assignment later becomes impossible, the assignment is removed and another candidate is tried. This early pruning avoids unnecessary search.

2. Input and Output

Item	Description
Input	A partially filled 9×9 grid; empty cells are represented by 0.
Output	A completed 9×9 grid satisfying all Sudoku constraints.
No solution	Report that no valid solution exists.

3. Sudoku Constraints

Row constraint: every row must contain digits 1–9 exactly once.

Column constraint: every column must contain digits 1–9 exactly once.

Subgrid constraint: every 3×3 box must contain digits 1–9 exactly once.

Given values are fixed and cannot be modified.

4. Backtracking Approach

- Find an empty cell.
- Try digits 1 through 9.

3. Check the row, column, and corresponding 3×3 box.
4. If valid, place the digit and recursively solve the remaining cells.
5. If the recursive call fails, reset the cell to 0 and try the next digit.
6. If no empty cell remains, the puzzle is solved.

5. Pseudocode

ALGORITHM SolveSudoku(Grid)

FUNCTION Solve(Grid)

 cell $\leftarrow$ FindEmptyCell(Grid)

 IF cell does not exist THEN

 RETURN TRUE

 row $\leftarrow$ cell.row

 col $\leftarrow$ cell.column

 FOR number $\leftarrow$ 1 TO 9 DO

 IF IsValid(Grid, row, col, number) THEN

 Grid[row][col] $\leftarrow$ number

 IF Solve(Grid) = TRUE THEN

 RETURN TRUE

 Grid[row][col] $\leftarrow$ 0

 // Backtrack

 RETURN FALSE

FUNCTION IsValid(Grid, row, col, number)

 FOR j $\leftarrow$ 0 TO 8 DO

 IF Grid[row][j] = number THEN

 RETURN FALSE

 FOR i $\leftarrow$ 0 TO 8 DO

 IF Grid[i][col] = number THEN

 RETURN FALSE

 startRow $\leftarrow$ row - (row MOD 3)

 startCol $\leftarrow$ col - (col MOD 3)

 FOR i $\leftarrow$ startRow TO startRow + 2 DO

 FOR j $\leftarrow$ startCol TO startCol + 2 DO

```
    IF Grid[i][j] = number THEN  
        RETURN FALSE
```

```
RETURN TRUE
```

```
MAIN
```

```
    Read the  $9 \times 9$  grid
```

```
    IF Solve(Grid) THEN
```

```
        Print Grid
```

```
    ELSE
```

```
        Print "No solution exists"
```

```
END
```

6. Python Implementation

```
def find_empty(grid):
```

```
    for row in range(9):
```

```
        for col in range(9):
```

```
            if grid[row][col] == 0:
```

```
                return row, col
```

```
    return None
```

```
def is_valid(grid, row, col, num):
```

```
    for j in range(9):
```

```
        if grid[row][j] == num:
```

```
            return False
```

```
    for i in range(9):
```

```
        if grid[i][col] == num:
```

```
            return False
```

```
    start_row = row - row % 3
```

```
    start_col = col - col % 3
```

```
    for i in range(start_row, start_row + 3):
```

```
        for j in range(start_col, start_col + 3):
```

```
            if grid[i][j] == num:
```

```
                return False
```

```
    return True
```

```
def solve_sudoku(grid):
```

```
    empty = find_empty(grid)
```

```
    if empty is None:
```

```
        return True
```

```

row, col = empty

for num in range(1, 10):
    if is_valid(grid, row, col, num):
        grid[row][col] = num

    if solve_sudoku(grid):
        return True

    grid[row][col] = 0

return False

grid = []
for _ in range(9):
    grid.append(list(map(int, input().split()))))

if solve_sudoku(grid):
    for row in grid:
        print(*row)
else:
    print("No solution exists")

```

7. Sample Input

```

5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9

```

8. Sample Output

```

5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9

```

9. Pruning and Search-Space Reduction

A naive approach can consider up to 9 choices for each empty cell, giving a worst-case search space of $O(9^M)$, where M is the number of empty cells. Backtracking prunes a branch immediately when a candidate violates a row, column, or subgrid constraint. Therefore, most impossible partial boards are never expanded.

The algorithm can be improved further by selecting the empty cell with the fewest legal candidates first, known as the Minimum Remaining Values (MRV) heuristic. This exposes contradictions earlier and can substantially reduce practical execution time.

10. Correctness

Every number is placed only after `IsValid` confirms that it does not occur in the selected row, column, or 3×3 box. Hence every partial grid maintained by the algorithm satisfies all Sudoku constraints.

When no empty cells remain, all 81 cells are filled and all constraints have been preserved, so the result is a valid solution. The algorithm is complete because it systematically tries every candidate that is not ruled out by a valid constraint; therefore, if a solution exists, it will eventually be found.

11. Handling All Possible Cases

Solved grid: immediately return success when no empty cell exists.

Valid incomplete grid: recursively fill empty cells.

Several candidates: test candidates systematically.

Invalid branch: undo the most recent placement and continue.

Unsolvable grid: return failure after all candidates are exhausted.

Invalid initial clues: a practical implementation should validate the initial grid before solving.

12. Complexity Analysis

Measure	Complexity	Explanation
Worst-case time	$O(9^M)$	M is the number of empty cells; at most nine candidates can be considered per empty cell.
Validity check	$O(9)$	One row, one column and one 3×3 box are checked; for fixed Sudoku size this is constant.
Auxiliary space	$O(M)$	Recursion depth is at most the number of empty cells, apart from the fixed grid.

13. Advantages

Simple recursive design.

Invalid placements are rejected early.

Can find one solution or all solutions with a small modification.

Uses little auxiliary memory.

Applicable to many constraint-satisfaction problems.

14. Limitations

Worst-case running time is exponential.

Very difficult puzzles may require substantial search.

Basic cell selection is not always optimal.

Advanced heuristics or bit-mask representations may be required for faster solving.

15. DAA Perspective

Sudoku demonstrates the backtracking paradigm in Design and Analysis of Algorithms. A candidate solution is constructed incrementally, feasibility is checked at each step, and an invalid partial solution is discarded before further recursion. The main performance improvement comes from pruning the state-space tree rather than eliminating the exponential worst-case nature of the problem.

16. Conclusion

The Sudoku Solver can be effectively designed using backtracking. It accepts a partially filled grid and fills empty cells while satisfying row, column, and 3×3 subgrid constraints. Invalid placements are skipped immediately, which greatly reduces unnecessary search.

The algorithm is logically sound and complete: it produces a valid completed grid whenever one exists and reports failure when no solution is possible. Although the worst-case complexity is exponential, pruning and heuristics such as MRV make backtracking practical for standard